14

Creating a Frontend to Consume and Send Events

After successfully creating a Socket.IO backend in the previous chapter, and doing our first experiments with the Socket.IO client, let's now focus on implementing a frontend to connect to the backend and consume and send events.

We are first going to clean up our project by removing files from the previously created blog app. Then, we are going to implement a React Context to initialize and store our Socket.IO instance, making use of the existing `AuthProvider` to provide the token for authenticating with the backend. After that, we are going to implement an interface for our chat app and a way to send chat messages, as well as displaying received chat messages. Finally, we are going to implement chat commands with acknowledgments to show which rooms we are currently in.

In this chapter, we are going to cover the following main topics:

- Integrating the Socket.IO client with React
- Implementing chat functionality
- Implementing chat commands with acknowledgments

Technical requirements

Before we start, please install all the requirements from ***Chapter 1***, *Preparing for Full-Stack Development*, and ***Chapter 2***, *Getting to Know*

Node.js and MongoDB.

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you have an issue with the code and steps provided in this book, please try using the versions listed in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub: **https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch14**.

If you cloned the full repository for the book, Husky may not find the **`.git`** directory when running **`npm install`**. In that case, just run **`git init`** in the root of the corresponding chapter folder.

The CiA video for this chapter can be found at: **https://youtu.be/d_TZK6S_XDU**.

Integrating the Socket.IO client with React

Let's start by cleaning up the project and deleting all old files copied over from the blog app. Then, we are going to set up a Socket.IO context to make it easier to initialize and use Socket.IO in React components. Finally, we are going to create our first component that utilizes this context to show the status of our Socket.IO connection.

Cleaning up the project

Let's first delete the folders and files from the blog application we created earlier:

1. Copy the existing **`ch13`** folder to a new **`ch14`** folder, as follows:

```
$ cp -R ch13 ch14
```

2. Open the **ch14** folder in VS Code.
3. *Delete* the following folders and files, as they were only required for the blog application backend:
    1. **backend/src/__tests__/**
    2. **backend/src/example.js**
    3. **backend/src/db/models/post.js**
    4. **backend/src/routes/posts.js**
    5. **backend/src/services/posts.js**
4. In **backend/src/app.js**, *remove* the following import:

```
import postRoutes from './routes/posts.js'
```

5. Also, *remove* **postRoutes**:

```
postRoutes(app)
```

6. *Delete* the following folders and files, as they were only required for the blog application frontend:
    1. **src/api/posts.js**
    2. **src/components/CreatePost.jsx**
    3. **src/components/Post.jsx**
    4. **src/components/PostFilter.jsx**
    5. **src/components/PostList.jsx**
    6. **src/components/PostSorting.jsx**
    7. **src/pages/Blog.jsx**

Now that we have cleaned up our project, let's get started with implementing a Socket.IO context for our new chat app.

Creating a Socket.IO context

Up until now, we have been initializing the Socket.IO client instance in the **src/App.jsx** component. However, doing this has some downsides:

- To access the socket in other components, we would need to pass it down via props.

- We can only have one socket connection for the whole app.
- It is not possible to get the token dynamically from `AuthContext`, requiring us to store the token in local storage instead.
- Our app requires a full refresh to be able to load the new token and connect with it.
- We still try to connect and get an error when not logged in.

To solve these issues, we can instead create a Socket.IO context. We can then use the provider component to do the following:

- Connect to Socket.IO only when the token is available in `AuthContext`.
- Store the status of the Socket.IO connection and use it within components to, for example, only show the chat interface when logged in.
- Store the error object and display errors in the user interface.

The following diagram shows how the status of our connection will be tracked:

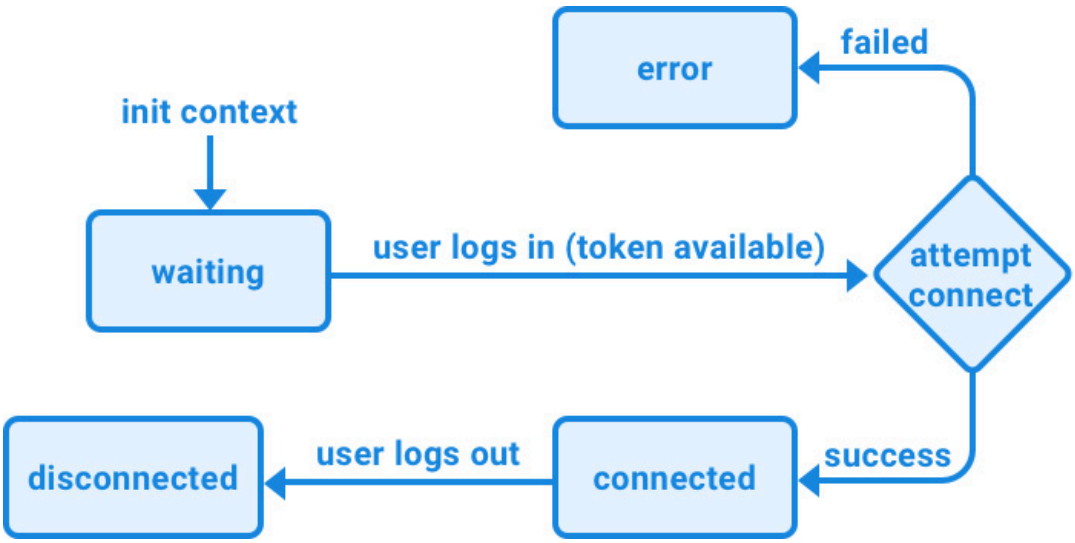

Figure 14.1 – The different states of the connection

As can be seen, the socket connection is initially waiting for the user to log in. Once the token is available, we attempt to establish a socket connection. If successful, the status changes to `connected`, otherwise to `error`. If the socket disconnects (for example, when the internet connection is lost), the state is set to `disconnected`.

Now, let's get started with creating a Socket.IO context:

1. Create a new **`src/contexts/SocketIOContext.jsx`** file.
2. Inside this file, import the following functions from **`react`**, **`socket.io-client`**, and **`prop-types`**:

```
import { createContext, useState, useContext, useEffect } from 'react'
import { io } from 'socket.io-client'
import PropTypes from 'prop-types'
```

3. Additionally, import the **`useAuth`** hook from **`AuthContext`** to get the current token:

```
import { useAuth } from './AuthContext.jsx'
```

4. Now, define a React Context with some initial values for **`socket`**, **`status`** and **`error`**:

```
export const SocketIOContext = createContext({
  socket: null,
  status: 'waiting',
  error: null,
})
```

5. Next, define a provider component, in which we first create state hooks for the different values of the context:

```
export const SocketIOContextProvider = ({ children }) => {
  const [socket, setSocket] = useState(null)
  const [status, setStatus] = useState('waiting')
  const [error, setError] = useState(null)
```

6. Then, use the **`useAuth`** hook to get the JWT (if available):

```
  const [token] = useAuth()
```

7. Create an effect hook that checks whether the token is available, and if so, attempts to connect to the Socket.IO backend:

```
  useEffect(() => {
    if (token) {
      const socket = io(import.meta.env.VITE_SOCKET_HOST, {
```

```
          query: window.location.search.substring(1),
          auth: { token },
        })
```

Just like before, we pass the host, the **`query`** string, and the **`auth`** object. However, now we get the token from the **`useAuth`** hook instead of local storage.

8. Create handlers for the **`connect`**, **`connect_error`**, and **`disconnect`** events and set the **`status`** string and the **`error`** object, respectively:

```
        socket.on('connect', () => {
          setStatus('connected')
          setError(null)
        })
        socket.on('connect_error', (err) => {
          setStatus('error')
          setError(err)
        })
        socket.on('disconnect', () => setStatus('disconnected'))
```

9. Set the **`socket`** object and list all necessary dependencies for the effect hook:

```
        setSocket(socket)
      }
    }, [token, setSocket, setStatus, setError])
```

10. Now we can return the provider, passing all values from the state hooks to it:

```
    return (
      <SocketIOContext.Provider value={{ socket, status, error }}>
        {children}
      </SocketIOContext.Provider>
    )
  }
```

11. Finally, we set **`PropTypes`** for the context provider component and define a **`useSocket`** hook that will simply return the whole context:

```
SocketIOContextProvider.propTypes = {
  children: PropTypes.element.isRequired,
}
export function useSocket() {
  return useContext(SocketIOContext)
}
```

Now that we have a context to initialize our Socket.IO client, let's hook it up and display the status of the socket connection.

Hooking up the context and displaying the status

We can now remove the code to connect to Socket.IO from the **`App`** component and use the provider instead, as follows:

1. Edit **`src/App.jsx`** and *remove* the following import:

```
import { io } from 'socket.io-client'
```

2. Add an import to **`SocketIOContextProvider`**:

```
import { SocketIOContextProvider } from './contexts/SocketIOContext.jsx'
```

3. Then, *remove* the following code related to the Socket.IO connection:

```
const socket = io(import.meta.env.VITE_SOCKET_HOST, {
  query: window.location.search.substring(1),
  auth: {
    token: window.localStorage.getItem('token'),
  },
})
socket.on('connect', async () => {
  console.log('connected to socket.io as', socket.id)
  socket.emit('chat.message', 'hello from client')
  const userInfo = await socket.emitWithAck('user.info', socket.id)
  console.log('user info', userInfo)
})
```

```
socket.on('connect_error', (err) => {
  console.error('socket.io connect error:', err)
})
socket.on('chat.message', (message) => {
  console.log(message)
})
```

4. Inside the **`App`** component, render the context provider:

```
export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthContextProvider>
        <SocketIOContextProvider>
          <RouterProvider router={router} />
        </SocketIOContextProvider>
      </AuthContextProvider>
    </QueryClientProvider>
  )
}
```

After hooking up the Socket.IO context, let's move on to creating a **`Status`** component to display the status.

Creating a Status component

Now, let's create a **`Status`** component to display the current status of the socket:

1. Create a new **`src/components/Status.jsx`** file.
2. Inside it, import the **`useSocket`** hook from our **`SocketIOContext`**:

```
import { useSocket } from '../contexts/SocketIOContext.jsx'
```

3. Define a **`Status`** component, in which we get the **`status`** string and **`error`** object from the hook:

```
export function Status() {
  const { status, error } = useSocket()
```

4. Render the socket status:

```
  return (
    <div>
      Socket status: <b>{status}</b>
```

5. If we have an **`error`** object, we can additionally display the error message now:

```
      {error && <i> - {error.message}</i>}
    </div>
  )
}
```

Now that we have a **`Status`** component, let's create a **`Chat`** page component, where we render the **`Header`** and **`Status`** components.

Creating a Chat page component

We previously had a **`Blog`** page for our blog app, which we deleted earlier in this chapter. Let's now create a new **`Chat`** page component for our chat app:

1. Create a new **`src/pages/Chat.jsx`** file.
2. Inside it, import the **`Header`** component (which we are going to reuse from the **`Blog`** app) and the **`Status`** component:

```
import { Header } from '../components/Header.jsx'
import { Status } from '../components/Status.jsx'
```

3. Render a **`Chat`** component in which we display the **`Header`** and **`Status`** components:

```
export function Chat() {
  return (
    <div style={{ padding: 8 }}>
      <Header />
      <br />
```

```
        <hr />
        <br />
        <Status />
      </div>
    )
  }
```

4. Edit **`src/App.jsx`** and locate the following import:

```
import { Blog } from './pages/Blog.jsx'
```

   Replace it with an import to the **`Chat`** component:

```
import { Chat } from './pages/Chat.jsx'
```

5. Finally, *replace* the **`<Blog />`** component in the main path in our router with the **`<Chat />`** component:

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Chat />,
  },
```

Starting and testing our chat app frontend

We can now start and test out our chat app frontend:

1. Run the frontend, as follows:

```
$ npm run dev
```

2. Run the backend, as follows (make sure Docker and the database container are running!):

```
$ cd backend/
$ npm run dev
```

3. Now go to **`http://localhost:5173/`** and you should see the following interface:

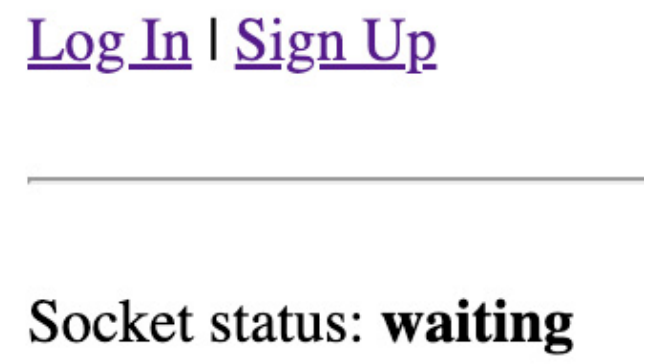

Figure 14.2 – Socket connection waiting for user to be logged in

4. Log in (create a new user if you do not have one yet), and the socket should connect successfully:

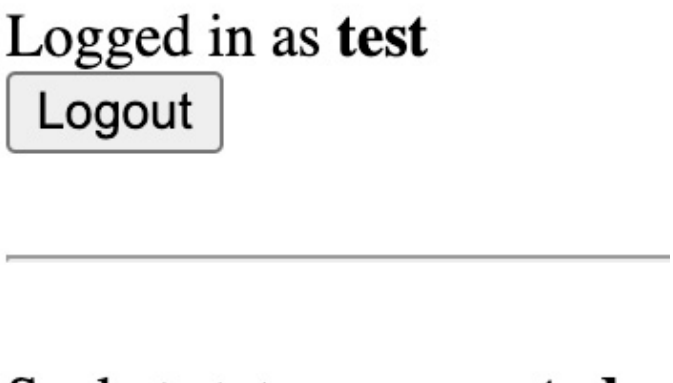

Figure 14.3 – Socket connected after user is logged in

Disconnecting socket on logout

You may have noticed that when pressing **Logout**, the socket stays connected. Let's fix that now, by disconnecting the socket when logging out:

1. Edit **`src/components/Header.jsx`** and import the **`useSocket`** hook:

```
import { useSocket } from '../contexts/SocketIOContext.jsx'
```

2. Get the socket from the **`useSocket`** hook, as follows:

```
export function Header() {
  const [token, setToken] = useAuth()
  const { socket } = useSocket()
```

3. Define a new **`handleLogout`** function, which disconnects the socket and resets the token:

```
const handleLogout = () => {
  socket.disconnect()
  setToken(null)
}
```

4. Lastly, set the **`onClick`** handler to the **`handleLogout`** function:

```
<button onClick={handleLogout}>Logout</button>
```

Now, when you log out, the socket will be disconnected, as can be seen in the following screenshot:

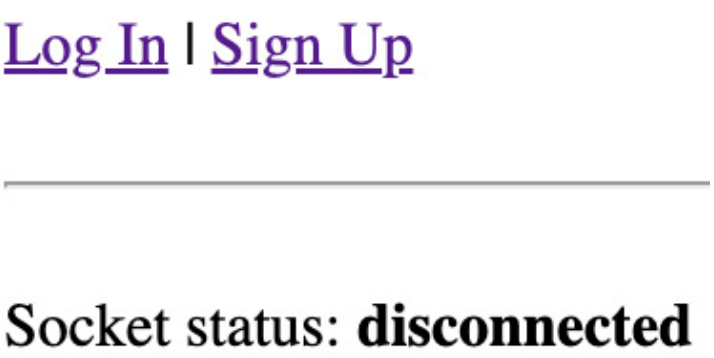

Figure 14.4 – Socket disconnected after logging out

Now that the Socket.IO client is successfully integrated with our React frontend, we can continue by implementing chat functionality in the frontend.

Implementing chat functionality

We are now going to implement functionality to send and receive messages in our chat app. First, we are going to implement all the components that we need. Then, we are going to create a **`useChat`** hook to implement the logic to interface with the socket connection and provide functions to send/receive messages. Lastly, we are going to put it all together by creating a chat room.

Implementing the chat components

We are going to implement the following chat components:

- **`ChatMessage`**: To display chat messages
- **`EnterMessage`**: A field to enter new messages and a button to send them

Implementing the ChatMessage component

Let's start by implementing the **`ChatMessage`** component:

1. Create a new **`src/components/ChatMessage.jsx`** file, which will render a chat message.
2. Import **`PropTypes`** and define a new function with **`username`** and **`message`** props:

```
import PropTypes from 'prop-types'
export function ChatMessage({ username, message }) {
```

3. Render the username in bold and the message next to it:

```
  return (
    <div>
      <b>{username}</b>: {message}
    </div>
  )
}
```

4. Define the prop types, as follows:

```
ChatMessage.propTypes = {
  username: PropTypes.string.isRequired,
  message: PropTypes.string.isRequired,
}
```

Implementing the EnterMessage component

Now, let's create the **`EnterMessage`** component, which will allow users to send a new chat message:

1. Create a new **`src/components/EnterMessage.jsx`** file.
2. Import the **`useState`** hook and **`PropTypes`**:

```
import { useState } from 'react'
import PropTypes from 'prop-types'
```

3. Define a new **`EnterMessage`** component, which receives an **`onSend`** function as props:

```
export function EnterMessage({ onSend }) {
```

4. We store the current state of the message entered:

```
  const [message, setMessage] = useState('')
```

5. Then, we define a function to handle sending the request and clearing the field afterward:

```
  function handleSend(e) {
    e.preventDefault()
    onSend(message)
    setMessage('')
  }
```

REMINDER

Because we are submitting a form using a **`submit`** *button, we need to call* **`e.preventDefault()`** *to prevent the form from refreshing the page.*

6. Render a form with an input field to enter the message and a button to send it:

```
  return (
    <form onSubmit={handleSend}>
      <input
```

```
        type='text'
        value={message}
        onChange={(e) => setMessage(e.target.value)}
      />
      <input type='submit' value='Send' />
    </form>
  )
}
```

7. Define the prop types, as follows:

```
EnterMessage.propTypes = {
  onSend: PropTypes.func.isRequired,
}
```

Implementing a useChat hook

To bundle all the logic together, we are going to implement a **`useChat`** hook, which is going to deal with sending and receiving messages, as well as storing all current messages in a state hook. Follow these steps to implement it:

1. Create a new **`src/hooks/`** folder. Inside it, create a new **`src/hooks/useChat.js`** file.
2. Import the **`useState`** and **`useEffect`** hooks from React:

```
import { useState, useEffect } from 'react'
```

3. Import the **`useSocket`** hook from our context:

```
import { useSocket } from '../contexts/SocketIOContext.jsx'
```

4. Define a new **`useChat`** function, where we get the socket from the **`useSocket`** hook, and define a state hook to store an array of messages:

```
export function useChat() {
  const { socket } = useSocket()
  const [messages, setMessages] = useState([])
```

5. Next, define a **`receiveMessage`** function, which appends a new message to the array:

```
  function receiveMessage(message) {
    setMessages((messages) => [...messages, message])
  }
```

6. Now, create an effect hook, in which we create a listener using **`socket.on`**:

```
  useEffect(() => {
    socket.on('chat.message', receiveMessage)
```

7. We need to make sure to remove the listener again using **`socket.off`** when the effect hook unmounts, otherwise we might end up with multiple listeners when the component re-renders or unmounts:

```
    return () => socket.off('chat.message', receiveMessage)
  }, [])
```

8. Now, receiving messages should work fine. Let's move on to sending messages. To do this, we create a **`sendMessage`** function, which uses **`socket.emit`** to send the message:

```
  function sendMessage(message) {
    socket.emit('chat.message', message)
  }
```

9. Lastly, return the **`messages`** array and the **`sendMessage`** function so that we can use them in our components:

```
  return { messages, sendMessage }
}
```

Now that we have successfully implemented the **`useChat`** hook, let's use it!

Implementing the ChatRoom component

Finally, we can put it all together and implement a **ChatRoom** component. Follow these steps to get started:

1. Create a new **src/components/ChatRoom.jsx** file.
2. Import the **useChat** hook and the **EnterMessage** and **ChatMessage** components:

```
import { useChat } from '../hooks/useChat.js'
import { EnterMessage } from './EnterMessage.jsx'
import { ChatMessage } from './ChatMessage.jsx'
```

3. Define a new component, which gets the **messages** array and the **sendMessage** function from the **useChat** hook:

```
export function ChatRoom() {
  const { messages, sendMessage } = useChat()
```

4. Then, render the list of messages as **ChatMessage** components:

```
  return (
    <div>
      {messages.map((message, index) => (
        <ChatMessage key={index} {...message} />
      ))}
```

5. Next, render the **EnterMessage** component and pass the **sendMessage** function as the **onSend** prop:

```
      <EnterMessage onSend={sendMessage} />
    </div>
  )
}
```

6. Edit **src/pages/Chat.jsx** and import the **ChatRoom** component and the **useSocket** hook:

```
import { ChatRoom } from '../components/ChatRoom.jsx'
import { useSocket } from '../contexts/SocketIOContext.jsx'
```

7. Get the status from the **useSocket** hook in the **Chat** page component:

```
export function Chat() {
  const { status } = useSocket()
```

8. If the status is **`connected`**, we show the **`ChatRoom`** component:

```
  return (
    <div style={{ padding: 8 }}>
      <Header />
      <br />
      <hr />
      <br />
      <Status />
      <br />
      <hr />
      <br />
      {status === 'connected' && <ChatRoom />}
```

9. Now, go to **`http://localhost:5173/`** in your browser and log in with a username and password. The socket connects and the chat room is rendered. Enter a chat message and send it by pressing *Return/Enter* or by clicking the **Send** button. You will see that the message is received and displayed!
10. Open a second browser window and log in with a second user. Send another message there. You will see that the message is received by both users, as can be seen in the following screenshot:

Figure 14.5 – Sending and receiving messages from different users

Now that we have a basic chat app working, let's explore how we could implement chat commands using acknowledgments.

Implementing chat commands with acknowledgments

In addition to sending and receiving messages, chat apps often offer a way to send commands to the client and/or server. For example, we could send a **`/clear`** command to clear our local messages list. Or we could send a **`/rooms`** command to get a list of rooms that we are in. Follow these steps to implement chat commands:

1. Edit **`src/hooks/useChat.js`** and adjust the **`sendMessage`** function inside it. First, let's make it an **`async`** function:

```
  async function sendMessage(message) {
```

2. *Replace* the contents of the function with the following. We first check whether the message starts with a slash (**`/`**). If so, then we get the command by removing the slash and use a **`switch`** statement:

```
    if (message.startsWith('/')) {
      const command = message.substring(1)
      switch (command) {
```

3. For the **`clear`** command, we simply set the array of messages to an empty array:

```
        case 'clear':
          setMessages([])
          break
```

4. For the **`rooms`** command, we get the user info by using **`socket.emitWithAck`** and our own **`socket.id`**:

```
        case 'rooms': {
          const userInfo = await socket.emitWithAck('user.info', socket.id)
```

5. Then, we get the list of rooms, filtering out our own room (with the name of our **`socket.id`**) that we automatically join in Socket.IO:

```
          const rooms = userInfo.rooms.filter((room) => room !== socket.id)
```

6. We reuse the **`receiveMessage`** function to send a message from the server, telling us the rooms that we are in:

```
          receiveMessage({
            message: `You are in: ${rooms.join(', ')}`,
          })
          break
        }
```

   Note that we are not sending a username here, just a message. We will have to adapt the **`ChatMessage`** component to accommodate that later.

7. If we receive any other command, we show an error message:

```
        default:
          receiveMessage({
            message: `Unknown command: ${command}`,
          })
          break
      }
```

8. Otherwise (if the message did not start with a slash), we simply emit the chat message, as before:

```
    } else {
      socket.emit('chat.message', message)
    }
  }
```

9. Finally, edit **`src/components/ChatMessage.jsx`** and adapt the component to render a system message if no username was given:

```
export function ChatMessage({ username, message }) {
  return (
    <div>
      {username ? (
        <span>
          <b>{username}</b>: {message}
        </span>
      ) : (
        <i>{message}</i>
      )}
    </div>
  )
}
```

10. Do not forget to adjust **`PropTypes`** to make the username optional (by *removing* **`.isRequired`** from the **`username`** prop):

```
ChatMessage.propTypes = {
  username: PropTypes.string,
```

11. Go to **`http://localhost:5173/`** in your browser and try sending a couple messages. Then, type **`/clear`** and you will see all messages were

cleared. Next, type `/rooms` to get the list of rooms that you are in, as you can see in the following screenshot:

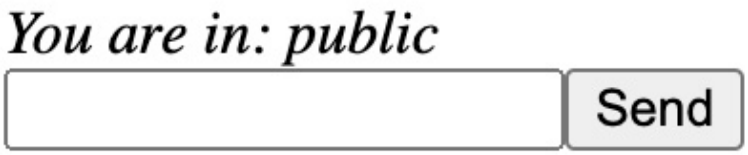

Figure 14.6 – Sending the /rooms command

NOTE

Joining different rooms currently does not work due to the query parameter getting cleared after logging in. In the next chapter, we are going to refactor the chat app and implement a `/join` *command to join a different room.*

Summary

In this chapter, we implemented a frontend for our chat app backend. We started by integrating the Socket.IO client with React by making a context and a custom hook for it. Then, we used `AuthProvider` to get the token to authenticate a user when connecting to the socket. After that, we displayed the status of our socket. Then, we implemented a chat app interface to send and receive messages. Finally, we implemented chat commands by using acknowledgments to get the rooms that we are in.

In the next chapter, ***Chapter 15***, *Adding Persistence to Socket.IO Using MongoDB*, we are going to learn how to store and replay previously sent messages using MongoDB with Socket.IO.